

PWM Breathing LED

PWM Breathing LED

- 1. PWM Breathing LED Overview
- 2. Core Concepts
 - 2.1 PWM Principle
 - 2.2 Breathing Algorithm: Triangular Wave
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code Details
 - 5.1 Macro Definitions and Global Variables
 - 5.2 `setup()` — Initialize PWM
 - 5.3 `loop()` — Breathing Algorithm
- 6. Operation Flow
 - 6.1 Build and Flash
 - 6.2 Normal Phenomena
 - 6.3 Verification Methods
- 7. Common Issues

1. PWM Breathing LED Overview

PWM (Pulse Width Modulation) controls LED brightness by changing the ratio of high/low level time. This experiment uses `analogWrite` on GPIO43 to generate PWM waves, changing the duty cycle in a triangular wave pattern to achieve an LED fade-in/fade-out "breathing" effect.

Typical Application Scenarios:

Application Type	Example
LED Control	Breathing light, keyboard backlight, ambient light brightness adjustment
Motor Drive	DC motor speed control, servo angle control, stepper motor micro-stepping
Audio Generation	Generate simple DAC / analog audio output with low-pass filter
Power Management	DC-DC converter control signals, charging IC enable modulation

Three Core Parameters of PWM

- ① **Period:** The time for one complete "on→off" cycle, in seconds. $\text{Period} = 1 / \text{Frequency}$.
- ② **Frequency:** How many "on→off" cycles per second, in Hz. How high should the frequency be?
 - **Too low (<50Hz):** Human eye can detect LED flickering, motor will shake
 - **Moderate (~1kHz):** LED appears stable, ordinary motor runs smoothly — this experiment uses 1kHz
 - **Too high (>20kHz):** Beyond human hearing range, suitable for quiet motor drive scenarios, but switching loss increases

③ **Duty Cycle:** In one cycle, the ratio of "high level time" to "entire cycle time".

$$\text{Duty Cycle} = (\text{High Level Time} / \text{Period}) \times 100\%$$

For example, frequency 1kHz (period = 1ms):

25% duty cycle = 0.25ms high + 0.75ms low → average voltage ≈ 0.825V

50% duty cycle = 0.50ms high + 0.50ms low → average voltage = 1.65V

80% duty cycle = 0.80ms high + 0.20ms low → average voltage = 2.64V

The beauty of PWM is: **you only change the "on/off" time ratio (duty cycle), but the load feels like the voltage is changing continuously.** This is the "area equivalent principle" — the average area value of a high-frequency square wave is equivalent to a constant analog voltage value.

2. Core Concepts

2.1 PWM Principle

PWM signal is determined by two parameters: **period** and **duty cycle**:

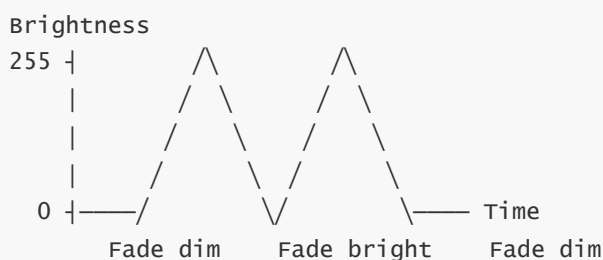
Period = 1/frequency (fixed, e.g. 5000Hz → 200μs)

Duty Cycle = High Level Time / Period × 100%

0% duty cycle: | _____ | – always low
50% duty cycle: | _____ | – half high half low
100% duty cycle: | _____ | – always high

ESP32-S3 uses LEDC (LED PWM Controller) peripheral, supporting up to 16-bit resolution. Arduino's `analogWrite` wraps LEDC, with duty cycle range 0~255 (8-bit effective).

2.2 Breathing Algorithm: Triangular Wave



Each iteration updates the brightness value by the step size, reverses direction downward when hitting the upper limit, and reverses direction upward when hitting the lower limit. `breathStep` controls the change speed (larger = faster but more stepping artifacts), `delayMs` controls the animation rhythm.

3. Hardware Dependencies

Pin	Peripheral	Configuration	Description
GPIO43	LED	OUTPUT	Common anode connection, <code>PWM=LOW</code> = on

With common anode connection, duty cycle is inversely related to brightness: `analogwrite(0)` = always low = LED brightest; `analogwrite(255)` = always high = LED off.

4. Project Architecture and Flow

```
setup()
├─ pinMode(43, OUTPUT)           // Configure GPIO as output
└─ analogwrite(43, 0)           // Initial duty cycle 0 = LED brightest

loop()
├─ brightness += direction * step // Update brightness by direction
├─ if brightness >= 255 → clamp + reverse direction (downward)
├─ if brightness <= 0   → clamp + reverse direction (upward)
├─ analogwrite(43, brightness)    // Output PWM
└─ delay(10ms)                   // Control change rate
```

5. Source Code Details

Complete source code: `Source Code` → `Arduino` → `1.Basic Peripherals` → `8.PWM_LED` → `8.PWM_LED.ino`

5.1 Macro Definitions and Global Variables

Three parameters control the breathing rhythm. `direction` of +1 means fade in, -1 means fade out.

```
#define LED0_PIN 43

const int breathStep = 5;           // Step size: change of 5 units per 10ms
const int delayMs = 10;            // Delay per step: 10ms
const int maxBrightness = 255;     // Maximum duty cycle

int currentBrightness = 0;          // Current brightness
int direction = 1;                 // Direction: +1 fade in, -1 fade out
```

Complete breathing cycle $\approx (256/5) \times 10\text{ms} \times 2 \approx 1024\text{ms} \approx 1$ second. Modify `breathStep` and `delayMs` to adjust speed.

5.2 `setup()` — Initialize PWM

`analogwrite(0)` sets initial duty cycle to 0, corresponding to LED brightest.

```
void setup() {  
  pinMode(LED0_PIN, OUTPUT);  
  analogwrite(LED0_PIN, 0);    // 0% duty cycle = always low = LED brightest  
}
```

After setting `pinMode` to `OUTPUT`, `analogwrite` automatically assigns LEDC channel and configures PWM parameters for that pin, no additional initialization needed.

5.3 `loop()` — Breathing Algorithm

Core of triangular wave brightness: reverse direction when hitting boundaries.

```
void loop() {  
  currentBrightness += direction * breathStep;  
  
  if (currentBrightness >= maxBrightness) {  
    currentBrightness = maxBrightness;  
    direction = -1;           // At top → reverse downward  
  } else if (currentBrightness <= 0) {  
    currentBrightness = 0;  
    direction = 1;           // At bottom → reverse upward  
  }  
  
  analogwrite(LED0_PIN, currentBrightness);  
  delay(delayMs);  
}
```

The purpose of clamping `currentBrightness` is to prevent the brightness from getting "stuck" outside the boundary when the step size does not divide evenly into 255.

6. Operation Flow

6.1 Build and Flash

Arduino flash process: See `Arduino → 1. Before Development → 2. Build and Flash`.

6.2 Normal Phenomena

After powering on, the LED cycles fade-in→fade-out with approximately 1 second period, simulating a breathing effect.

6.3 Verification Methods

Operation	Expected Result
Power on and observe	LED fades in and out smoothly, no flickering
Change <code>breathStep = 5</code> to <code>breathStep = 20</code>	Breathing speeds up, stepping artifacts more noticeable
Change <code>delayMs = 10</code> to <code>delayMs = 30</code>	Breathing slows down, smoother

7. Common Issues

Phenomenon	Cause	Solution
LED not lit	Incorrect <code>analogWrite</code> value (with common anode, 255=off)	Confirm initial value is 0 not 255
Obvious stepping artifacts in breathing	Step size too large or delay too short	Decrease <code>breathStep</code> or increase <code>delayMs</code>
Low PWM frequency causing flicker	Default LEDC frequency ~5kHz, not perceivable by eye	Normal, no adjustment needed